# Data Structures (CS2103)

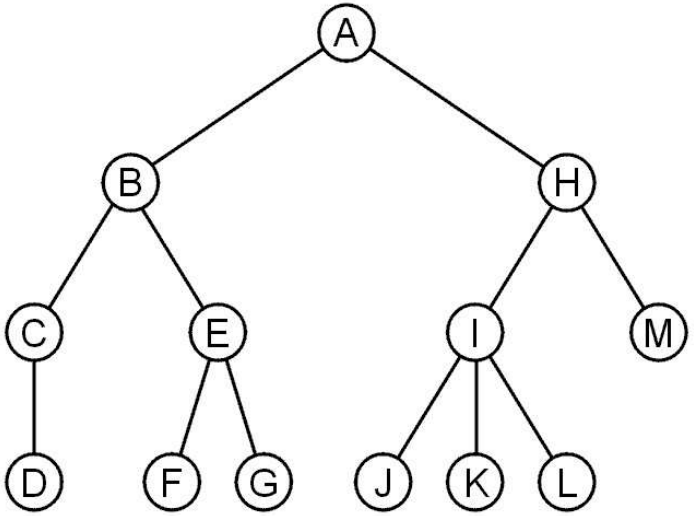
## Tree

**Prepared By – Ashish K Layek**

**CST, IIEST Shibpur**

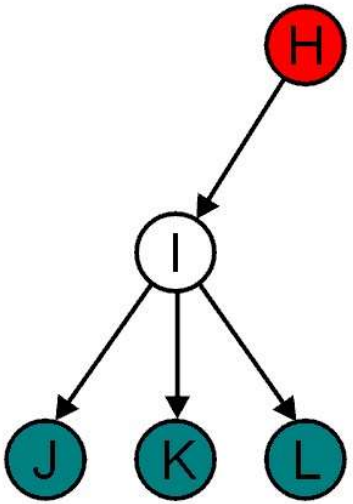
# Trees

- A rooted tree data structure stores information in *nodes*
  - There is a first node, or *root*
  - Each node has variable number of references to successors
  - Each node, other than the root, has exactly one node pointing to it



# Terminology: Child and Parent

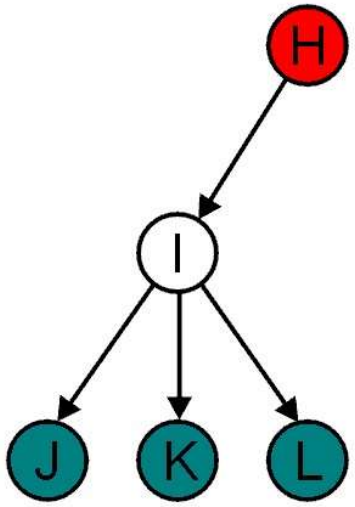
- All nodes will have zero or more child nodes or *children*
  - I has three children: J, K and L
- For all nodes other than the root node, there is one parent node
  - H is the parent I



# Terminology: Degree

- The *degree* of a node is defined as the number of its children: deg(I) = 3

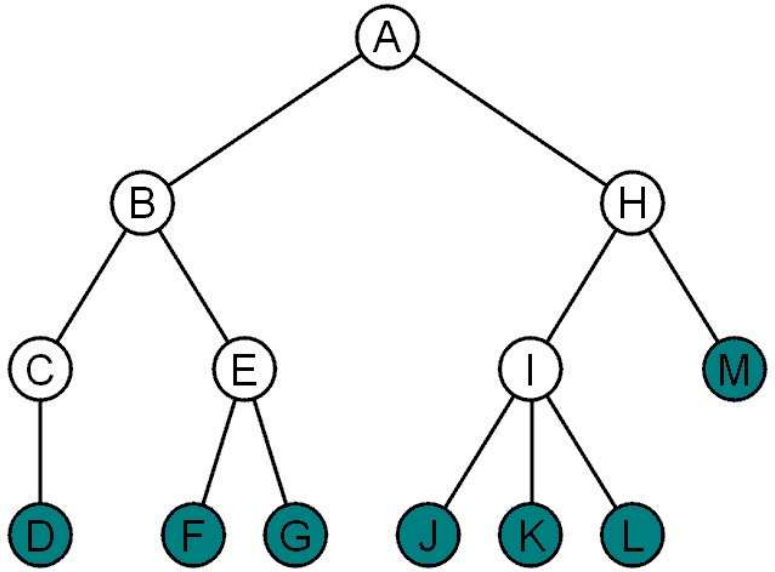
- Nodes with the same parent are *siblings*
  - J, K, and L are siblings



# Terminology: Leaf and Internal Node

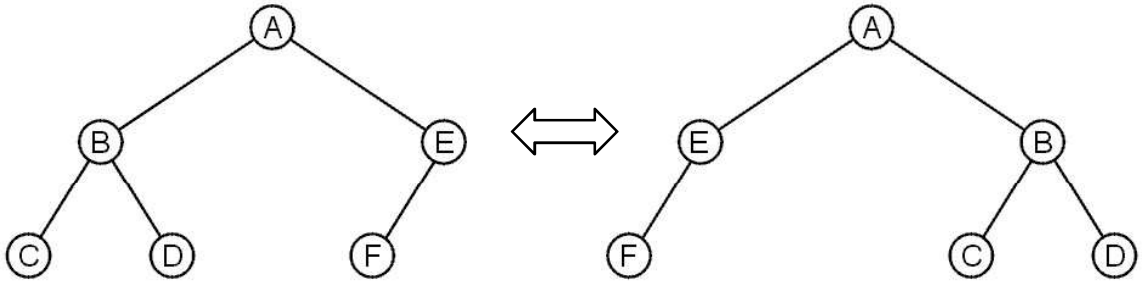
- Nodes with degree zero are also called *leaf nodes*

- All other nodes are said to be *internal nodes*, that is, they are internal to the tree



# Terminology: Order

- These trees are equal if the order of the children is ignored
  - *unordered trees*




- They are different if order is relevant (*ordered trees*)
  - We will usually examine ordered trees (linear orders)
  - In a hierarchical ordering, order is not relevant

# Terminology: Path

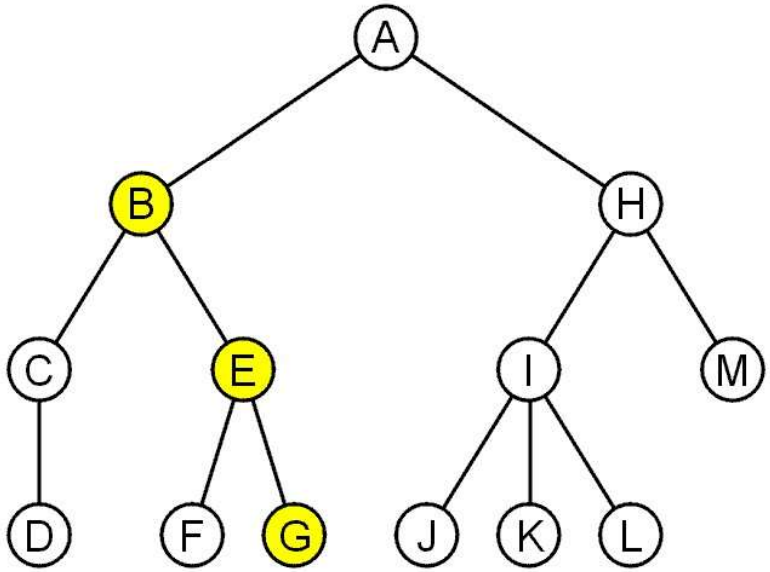
- A path is a sequence of nodes

  $(a_0, a_1, \ldots, a_n)$

  where $a_{k+1}$ is a child of $a_k$

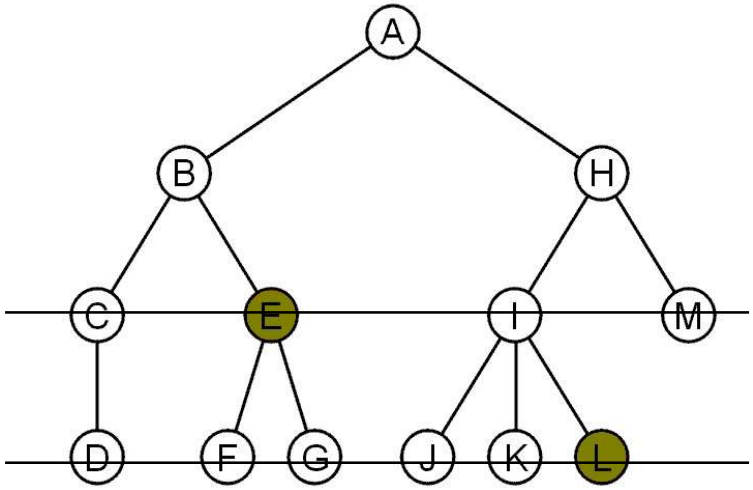
  The length of this path is $n$

  *E.g.*, the path (B, E, G) has length 2



# Terminology: Depth and Level

- The **level** and **depth** are closely related concepts, but they refer to slightly different things depending on perspective
- For each node in a tree, there exists a unique path from the root node to that node
- The length of this path is the *depth* of the node, *e.g.*,
  - E has depth 2 and L has depth 3
- The *level* of a node is defined by initially letting the root be at level ZERO. If a node is at level *l*, then its children are at level *l + 1*.
- In most cases, depth = level




*Note: Sometimes level of root is considered as 1, depending on convention. In that case depth and level differs in terms of value*

# Terminology: Height

- The *height* of a tree is defined as the maximum depth of any node within the tree

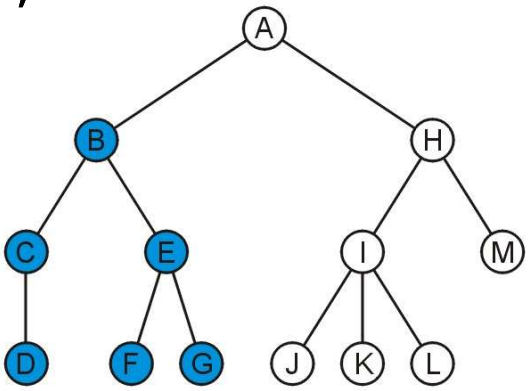
- The height of a tree with one node is 0
  - Just the root node

- For convenience, we define the height of the empty tree to be –1. (*Empty tree is possible in case of Binary Tree*)

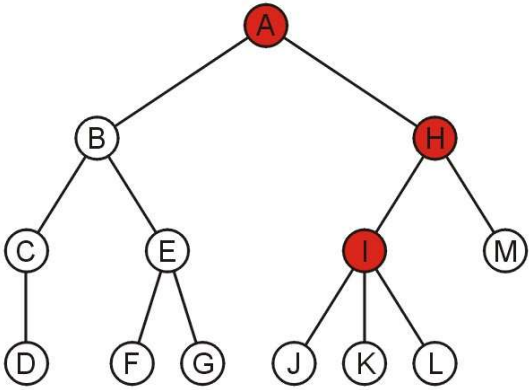
# Terminology: Ancestor and Descendent

- If a path exists from node *a* to node *b*:
  - *a* is an *ancestor* of *b*
  - *b* is a *descendent* of *a*
- The root node is an ancestor of all nodes

The descendants of node B are C, D, E, F, and G:




The ancestors of node I are H, and A:



# Tree definition

A tree is a finite set of one or more nodes such that (i) there is a specially designated node called the root; (ii) the remaining nodes are partitioned into n >= 0 disjoint sets ($T_1$, $T_2$, ..., $T_n$) where each of these sets is a tree. $T_1$, $T_2$, ..., $T_n$ are called the subtrees of the root.

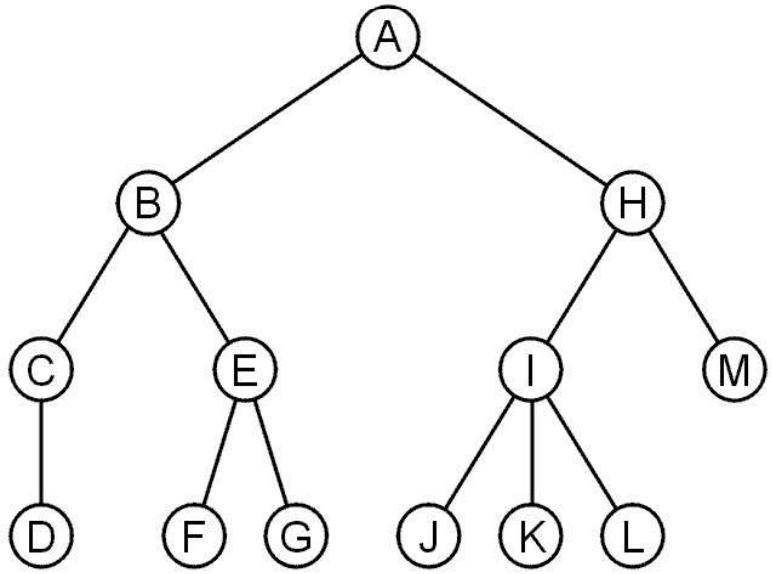
Let us look carefully at the definition and note down the important points.

1. the definition is recursive.
2. the set is finite and must have at least one node for it to be tree.
3. an empty set is not considered to be a tree.
4. the condition that $T_1$, $T_2$, ..., $T_n$ be disjoint sets prohibits subtrees from ever connecting together.
5. every node in the tree is root of some subtree.

# Abstract Trees

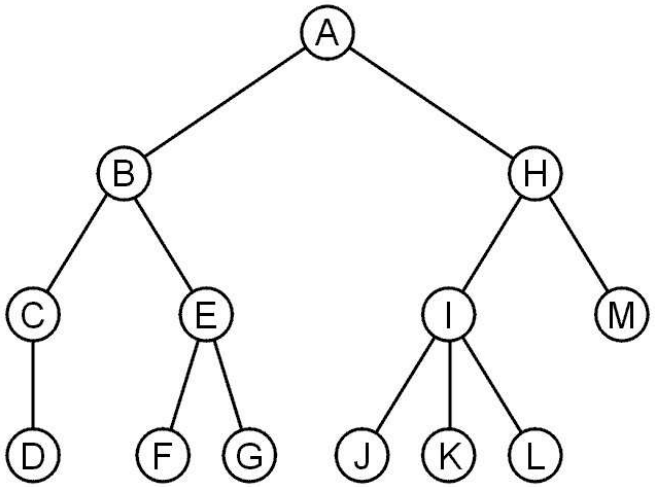
- An abstract tree (or abstract hierarchy) does not restrict the number of nodes
  - In this tree, the degrees vary:

| Degree | Nodes |
|---|---|
| 0 | D, F, G, J, K, L, M |
| 1 | C |
| 2 | A, B, E, H |
| 3 | I |



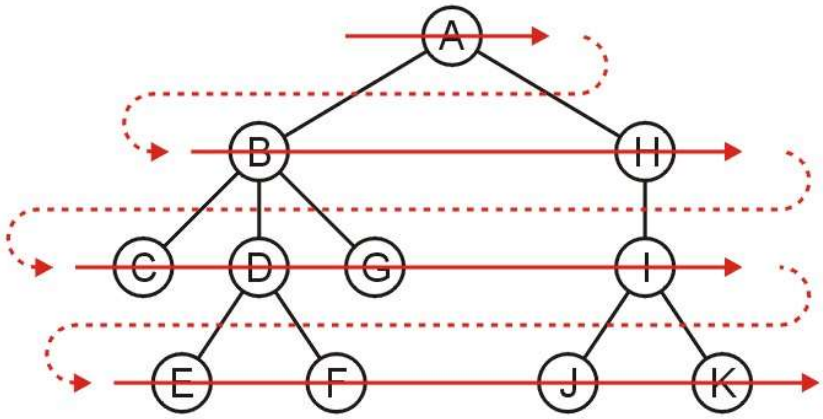
# Abstract Trees: Design

- We implement an abstract tree or hierarchy by using a structure that:
  - Stores a value
  - Stores the children in a linked-list

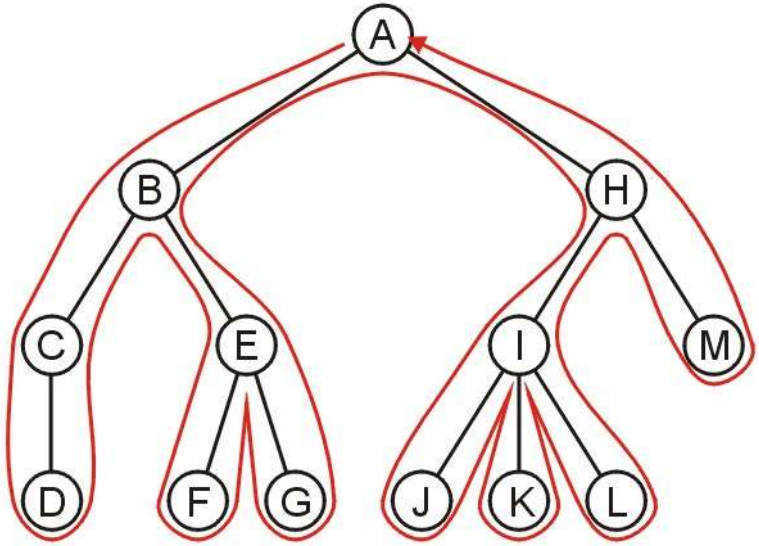


# Tree Traversal (for your study)

- Breadth-first traversal
  - Visits all nodes at depth *k* before proceeding onto depth *k* + 1

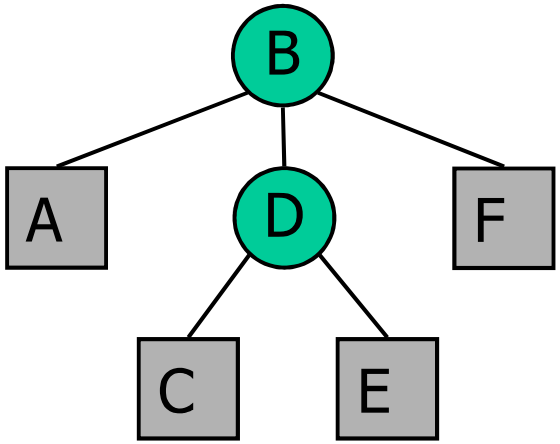


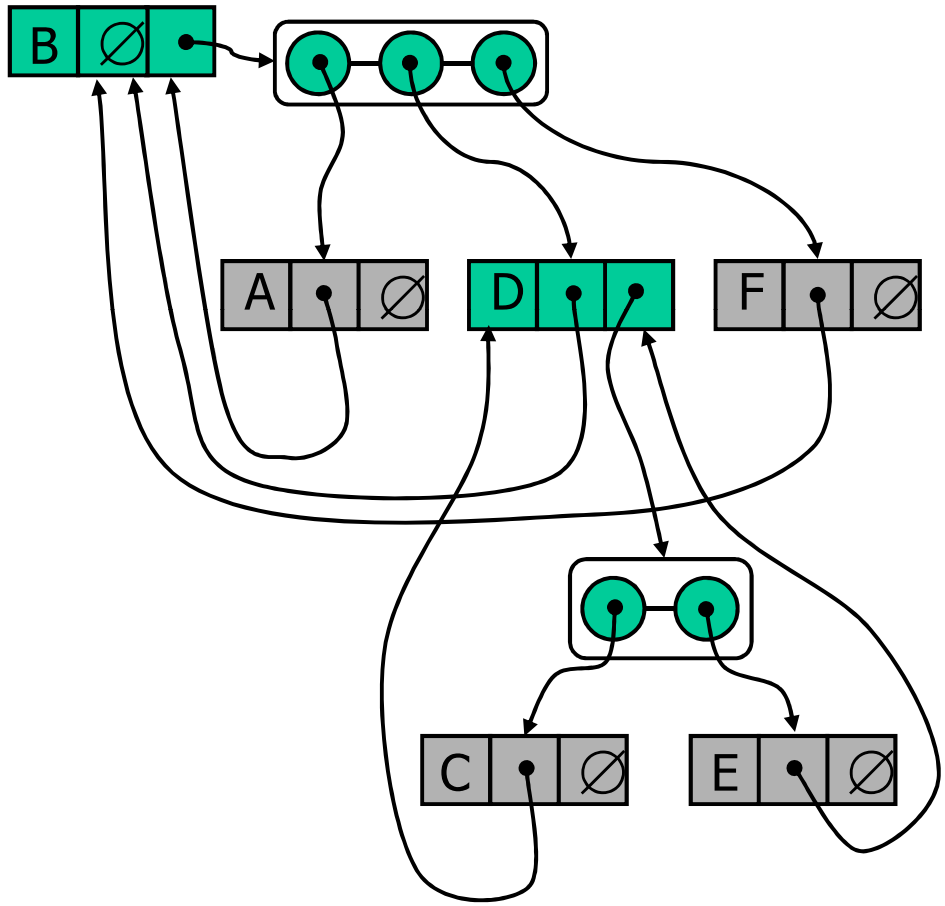

- Depth-first traversal
  - At any node, we proceed to the first child that has not yet been visited
  - Or, if we have visited all the children (of which a leaf node is a special case), we backtrack to the parent and repeat this decision making process



# Linked Structure for Trees

- A node is represented by an structure storing
  - Element
  - Parent node
  - Sequence of children nodes



# Linked Structure for Trees Contd …

```
// forward declaration
struct childNode;

struct treeNode {
    // data field
    char info;
    // reference to parent
    struct treeNode * parent;
    // list of childs
    struct childNode * childs;
};

struct childNode {
    struct treeNode child; // * ??
    struct childNode * next;
};
```

```
// An alternative approach
with limitations

struct treeNode {
    // data field
    char info;
    // reference to parent
    struct treeNode * parent;
    // list of childs can be
allocated according to the
tree structure – like number
of childs.
    struct treeNode ** childs;
};
```

# Binary Tree

# Motivation

- The arbitrary number of children in general trees is often unnecessary—many real-life trees are restricted to two branches
  - Expression trees using binary operators
  - An ancestral tree of an individual, parents, grandparents, *etc*.
  - Phylogenetic trees
  - Lossless encoding algorithms

- There are also issues with general trees:
  - There is no natural order between a node and its children

# Binary Tree definition

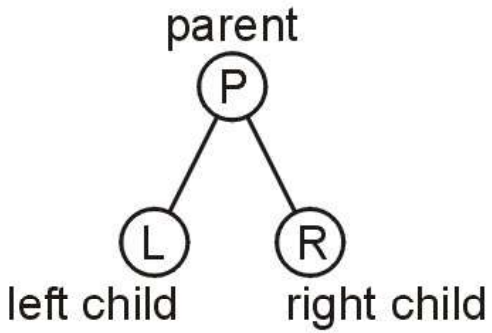
A binary tree is a finite set of nodes which is either empty or consists of a root and two disjoint binary trees called the left subtree and the right subtree.

Few important observations to note from the definition

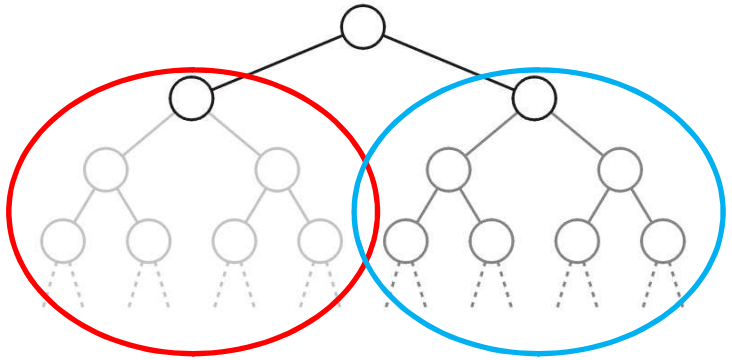
1. The definition is recursive.
2. Unlike tree, binary tree can be empty.
3. Order of the subtree does matter here, i.e., we distinguish between left subtree and right subtree.

# Explanation

- A binary tree is a restriction where each node has exactly two children:
  - Each child is either empty or another binary tree
  - This restriction allows us to label the children as *left* and *right* subtrees

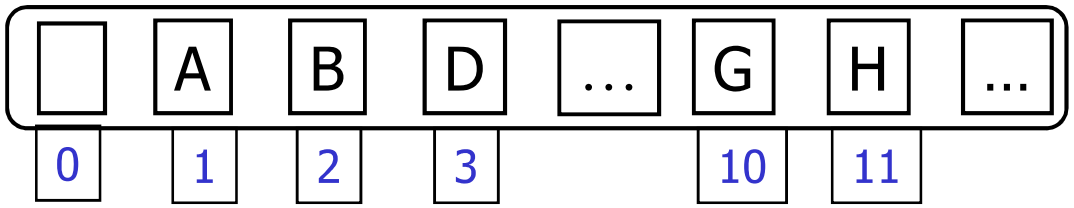



- We will also refer to the two sub-trees as
  - The left-hand sub-tree, and
  - The right-hand sub-tree

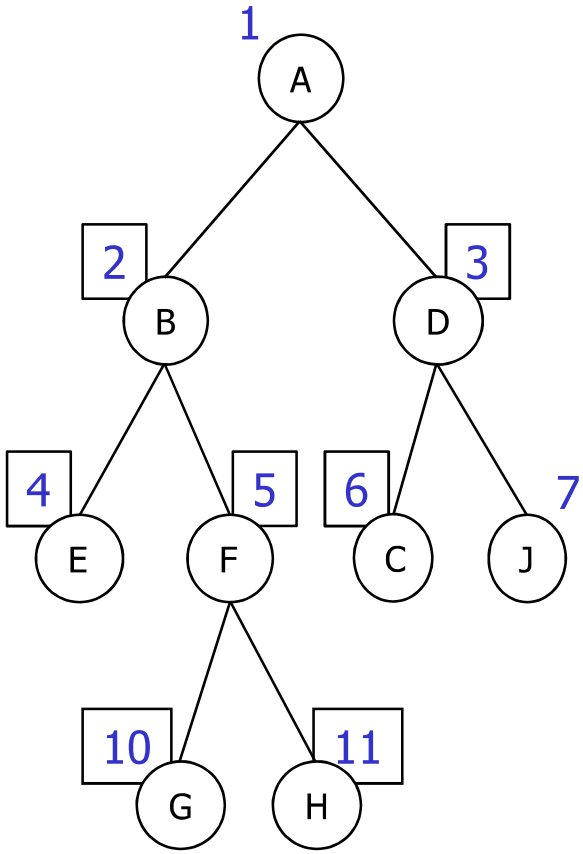


# Array-Based Representation of Binary Trees

- Nodes are stored in an array (*leaving first element unused in case base index of the array is 1*)

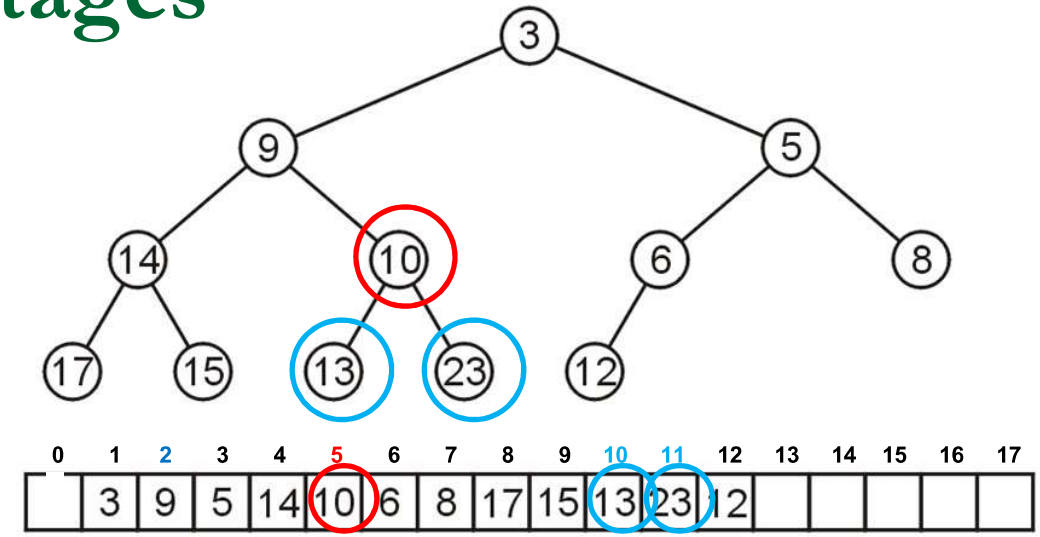



- The left child of node i can be found at position 2i
- The right child of node i can be found at position 2i + 1
- The parent of node i can be found at position i/2

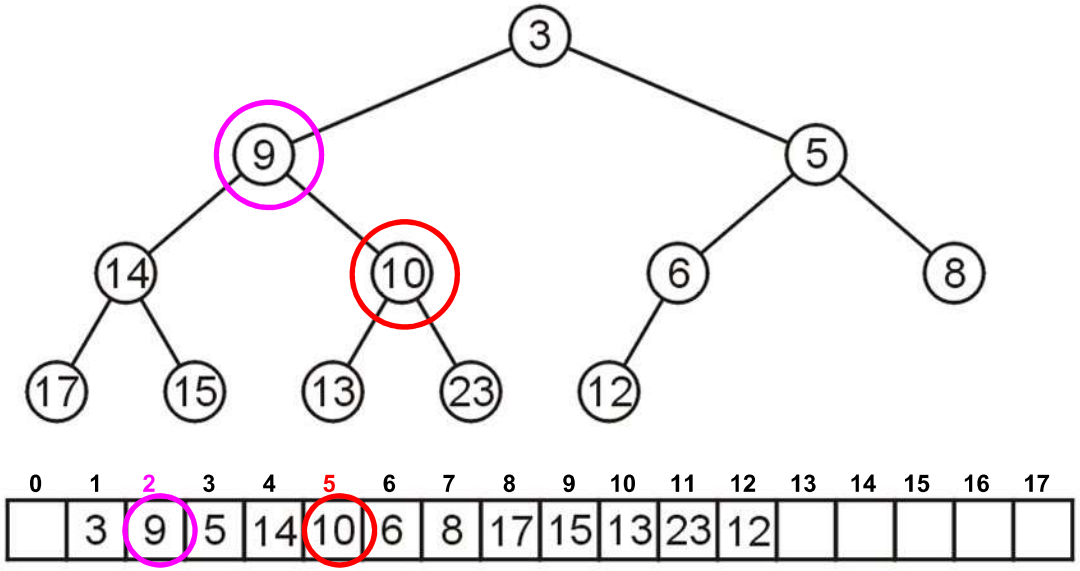


# Array storage: Advantages

- For example, node 10 has index 5:
  - Its children 13 and 23 have indices 10 and 11, respectively



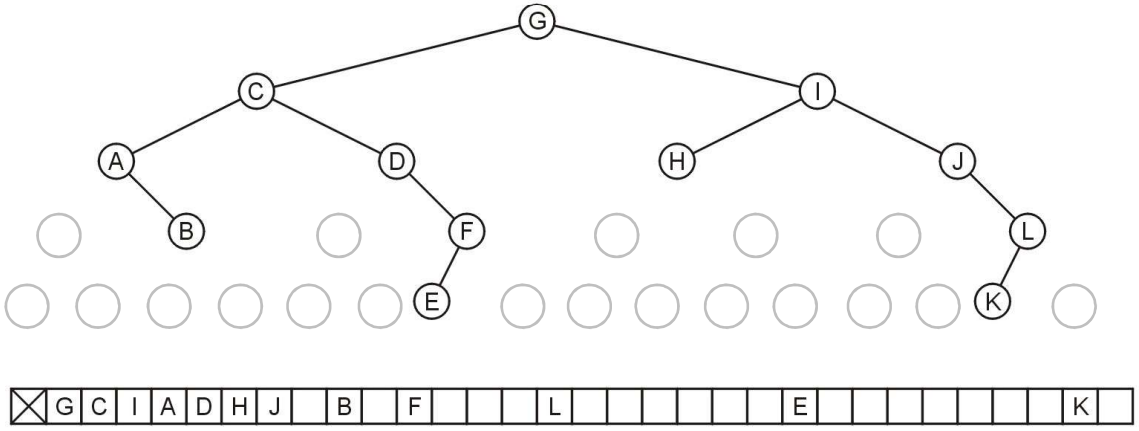

  - Its parent is node 9 with index 5/2 = **2**




- Conclusion: Constant time access to parent or child

# Array storage: Disadvantages

- There is a significant potential for a lot of wasted memory
  - Consider this tree with 12 nodes would require an array of size 32
    - ✓ Adding a child to node K doubles the required memory

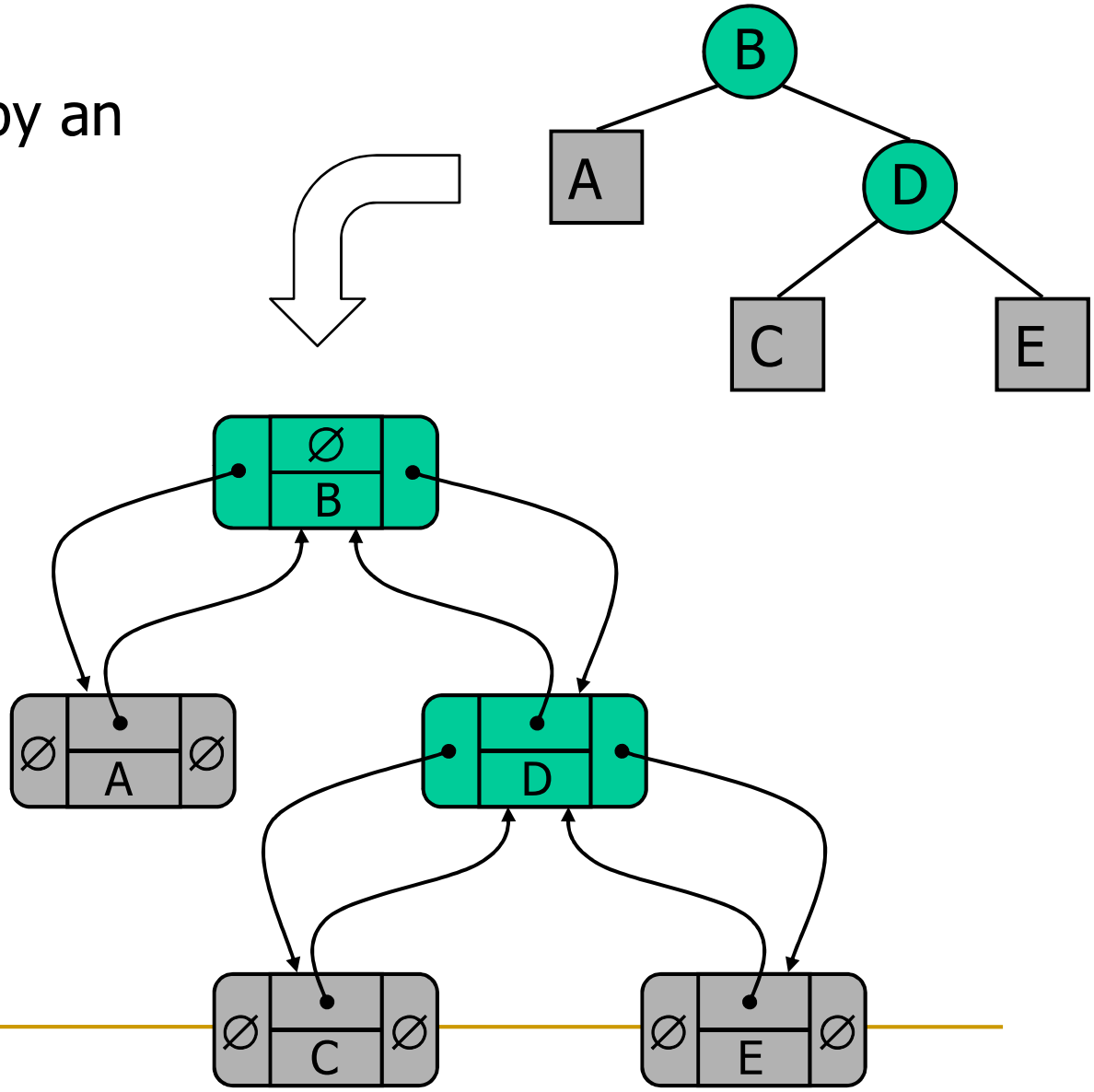



- Conclusion: Array based binary tree representation is suitable only for Perfect and Complete Binary Tree

# Linked Node Representation for Binary Trees

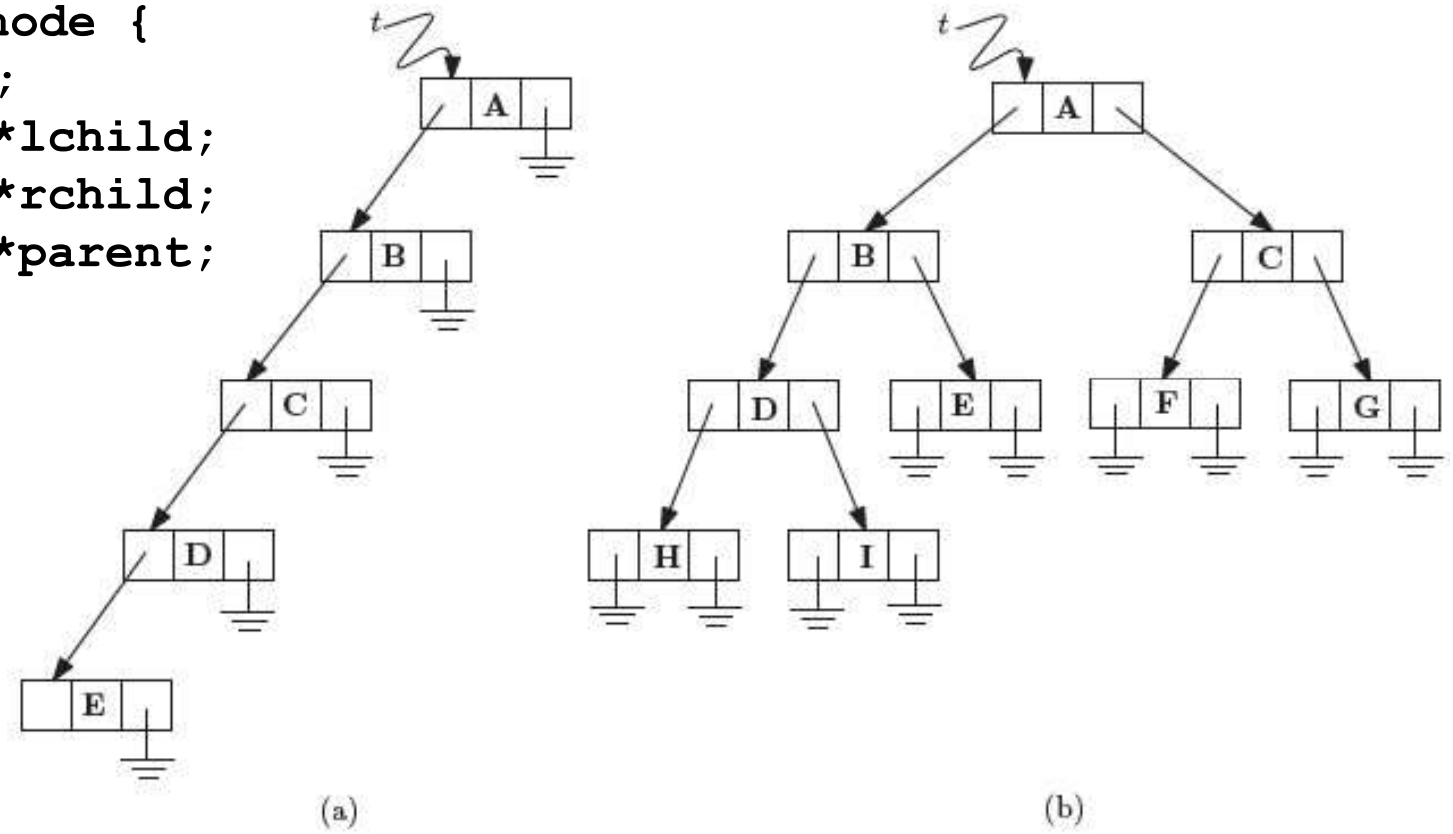
- A node is represented by an structure storing
  - Element
  - Parent node
  - Left child node
  - Right child node

```
typedef struct node {
    char element;
    struct node *lchild;
    struct node *rchild;
    struct node *parent;
} treenode;
```



# Examples

```
typedef struct node {
    char element;
    struct node *lchild;
    struct node *rchild;
    struct node *parent;
} treenode;
```




*Note: Arrows to Parent nodes are not shown in these diagrams*